什么是函数

1

什么是函数

- 假如我们要分别计算6的阶乘，还有10的阶乘：

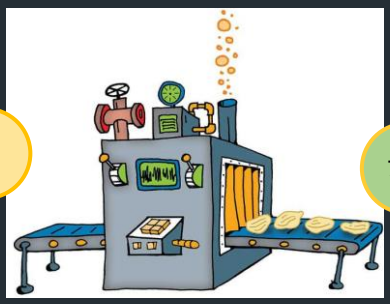
```
var result = 1;
for (var i = 1; i <= 6; i++) {
    result *= i;
}
console.log(result);

var result = 1;
for (var i = 1; i <= 10; i++) {
    result *= i;
}
console.log(result);
```

2

什么是函数

- 题目：请计算3! + 4! + 5! + 6! + 7!的和
- 计算阶乘是一个独立的操作，如果有一个计算阶乘的“机器”该多好啊

6



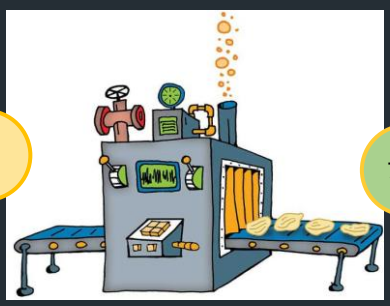
720

3

什么是函数

- 函数就是语句的封装，“一次定义，多次调用”
- 使用函数，可以简化问题，让问题规模变小

6



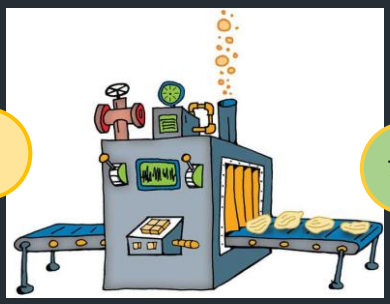
720

4

快速演示函数的使用

- 现在，老师进行demo演示，快速展示给你看到底如何使用函数
- 当然，有一些具体的知识可能你不是太懂，没事，这只是一个整体演示

6



720

5

函数的定义和调用

6

定义函数

- 和变量类似，函数必须先定义然后才能使用
- 使用function关键字定义函数，function是“功能”的意思

function是“功能”的意思，函数就是封装一定的“功能”

函数名，函数名必须符合JS标识符命名规则

圆括号中是形参列表，即使没有形参，也必须书写圆括号

```
function fun() {
    // 函数体语句
}
```

大括号中就是函数体语句

7

函数表达式

- 函数可以没有名字，而是赋值给一个变量，这种方法也可以定义函数，叫做“函数表达式”

匿名函数

```
var fun = function () {
    // 函数体语句
}
```

8

调用函数

- 执行函数体中的所有语句，就称为“调用函数”
- 调用函数非常简单，只需在函数名字后书写圆括号对即可

```
fun();        // 调用函数
```

圆括号中是实参列表，如果没有实参，也要书写圆括号

9

语句执行顺序

```
function fun() {
    console.log('A');
    console.log('B');
    console.log('C');
}
console.log(1);
console.log(2);
console.log(3);
fun();
console.log(4);
console.log(5);
console.log(6);
```

函数不调用内部语句就不会执行

执行函数

从这里开始执行

程序控制权交给函数

函数执行完毕，将控制权交回给主程序

```
1
2
3
A
B
C
4
5
6
```

10

函数的提升

11

函数的提升

- 和变量声明提升类似，函数声明也可以被提升
- 这意味着，即使先调用函数，然后再定义函数，函数也能执行，不会报错

在预解析阶段会被提升

```
fun();
function fun() {
    console.log('A');
}
```

12

函数表达式不能提升

- 如果函数是用函数表达式的写法定义的，则没有提升特性

只提升定义，不提升值，相当于这里在试图执行undefined

```
fun();            // 引发错误
var fun = function () {
    alert("函数被执行");
}
```

13

函数比变量先提升

函数先提升

变量声明后提升，这里var fun被提升，由于没有带着值提升，所以没有覆盖函数

```
fun(); 输出B
var fun = function () {
    console.log('A');
}
function fun() {
    console.log('B');
}
fun(); 输出A
```

程序执行阶段，会将函数覆盖fun变量

程序执行阶段，不会再次阅读函数的定义了

14

函数比变量先提升

```
fun();
var fun = function () {
    console.log('A');
}
function fun() {
    console.log('B');
}
fun();
```

15

函数的参数

16

函数的参数

- 参数是函数内的一些待定值，在调用函数时，必须传入这些参数的具体值
- 函数的参数可多可少，函数可以没有参数，也可以有多个参数，多个参数之间需要用逗号隔开

17

形式参数和实际参数

定义函数时，圆括号中定义“形式参数”

```
function add(a, b) {
    var sum = a + b;
    console.log('两个数字的和是' + sum);
}
add(3, 5);
```

调用函数时，圆括号中传入“实际参数”

18

“形实结合”

```
function add(a, b) {
    var sum = a + b;
    console.log('两个数字的和是' + sum);
}
add(3, 5);
```

19

形参和实参个数不同的情况

```
function add(a, b) {
    var sum = a + b;
    console.log('两个数字的和是' + sum);
}
add(3, 5, 9);
```

没有形参接收它

20

形参和实参个数不同的情况

undefined

```
function add(a, b) {
    var sum = a + b;
    console.log('两个数字的和是' + sum);
}
add(3);
```

21

函数的返回值

22

函数的返回值

- 函数体内可以使用return关键字表示“函数的返回值”

23

小案例：喇叭花数

24

小案例：喇叭花数

- 题目：喇叭花数是这样的三位数：其每一位数字的阶乘之和恰好等于它本身。即abc = a! + b! + c!，其中abc表示一个三位数。试寻找所有喇叭花数。
- 思路：将计算阶乘封装为函数，这样问题就被“降维”了

25

小案例：回文质数

26

小案例：回文质数

- 题目：“回文质数”是指这个数字既是回文数，又是质数。比如数字313就是一个回文质数、数字2也是一个回文质数。请编程寻找前100个回文质数。
- 思路：封装两个函数，第一个函数判断这个数字是不是回文的；第二个函数判断这个数字是不是质数。

27

小案例：互反质数

28

小案例：互反质数

- 题目：互反质数是指它本身是质数，它的逆序数字也是质数。比如17和71；13和31等。请输出前100个互反质数，要求按从小到大输出，每行显示10个，并且准确对齐，比如：

```
13    17    31    37    71    73    79    97    107   113
149   157   167   179   199   311   337   347   359   389
……
```

- 这个题目是某大厂（忘记了）校园招聘题目

29

全局变量和局部变量

30

变量作用域

- JavaScript是函数级作用域编程语言：变量只在其定义时所在的function内部有意义。

```
function fun() {
    var a = 10;
}
fun();
console.log(a);     // 报错
```

变量a是在fun函数中被定义的，所以变量a只在fun函数内部有定义，fun函数就是a的“作用域”，变量a被称为局部变量。

31

全局变量

- 如果不将变量定义在任何函数的内部，此时这个变量就是全局变量，它在任何函数内都可以被访问和更改。

```
var a = 10;
function fun() {
    a++;
    console.log(a);     //  输出11
}
fun();
console.log(a);         // 输出11
```

变量a没有定义在任何函数内部，它是“全局变量”

32

遮蔽效应

- 如果函数中也定义了和全局同名的变量，则函数内的变量会将全局的变量“遮蔽”

```
var a = 10;
function fun() {
    var a = 5;
    a++;
    console.log(a); //  输出6
}
fun();
console.log(a);     // 输出10
```

局部变量a将全局变量a“遮蔽”了

改变的是局部变量a的值，不影响全局变量

33

注意考虑变量声明提升的情况

- 这个程序的运行结果是什么呢？

```
var a = 10;
function fun() {
    a++;
    var a = 5;
    console.log(a);
}
fun();
console.log(a);
```

局部变量a被自增1，a此时是undefined，自增1结果是NaN

重新将a赋值为5

```
5
10
```

34

形参也是局部变量

- 这个程序的运行结果是什么呢？

```
var a = 10;
function fun(a) {
    a++;
    console.log(a);
}
fun(7);
console.log(a);
```

形参a也是函数内部的局部变量

```
8
10
```

35

不加var定义的是全局变量

- 这个程序的运行结果是什么呢？

```
function fun() {
    a = 10;
}

fun();
console.log(a);     // 10
```

不加var给变量赋值，相当于定义了一个全局变量

36

变量作用域面试题举例

37

变量作用域面试题举例1

- 变量声明提升无视if语句

```
var a = false;
function fun() {
    a = 10;
    if (a) {
        var a = 20;
    }
}
fun();
console.log(a);
```

局部变量

程序预读取阶段，无论if语句是否为真，var a都能提升！

false

38

变量作用域面试题举例2

- 形参没有被传值，也能够遮蔽全局变量

```
var m = 10;
var n = 20;
function fun(m, n) {
    m += n;
    console.log(m);
}
fun(30);
console.log(m);
```

局部变量

```
NaN
10
```

39

作用域链

40

作用域链

- 先来认识函数的嵌套：一个函数内部也可以定义一个函数。和局部变量类似，定义在一个函数内部的函数是局部函数。

```
function fun() {
    function inner() {
        console.log('你好');
    }
    inner();
}
fun();
```

局部函数

41

作用域链

- 在函数嵌套中，变量会从内到外逐层寻找它的定义。

```
var a = 10;
var b = 20;
function fun() {
    var c = 30;
    function inner() {
        var a = 40;
        var d = 50;
        console.log(a, b, c, d);
    }
    inner();
}
fun();
```

使用变量时，JS会从当前层开始，逐层向上寻找定义

42

闭包

43

从一个题目开始

- 这个程序的运行结果是什么呢？

```
function fun() {
    var a = 10;
    function inner() {
        alert(a);
    }
    return inner;
}
var inn = fun();
inn();
```

内部函数被return返回到了外部

inn变量现在存储的就是内部函数

44

什么是闭包

- JavaScript中函数会产生闭包（closure）。闭包是函数本身和该函数声明时所处的环境状态的组合。

45

什么是闭包

- 函数能够“记住”其定义时所处的环境，即使函数不在其定义的环境中被调用，也能访问定义时所处环境的变量。

```
function fun() {
    var a = 10;
    function inner() {
        alert(a);
    }
    return inner;
}
var inn = fun();
inn();
```

10

46

观察闭包现象

- 在JavaScript中，每次创建函数时都会创建闭包
- 但是，闭包特性往往需要将函数“换一个地方”执行，才能被观察出来

47

闭包非常实用

- 闭包很有用，因为它允许我们将数据与操作该数据的函数关联起来，这与“面向对象编程”有少许相似之处
- 闭包的功能：记忆性、模拟私有变量。

48

闭包用途1 - 记忆性

- 当闭包产生时，函数所处环境的状态会始终保持在内存中，不会在外层函数调用后被自动清除。这就是闭包的记忆性。
- 题目：创建体温检测函数checkTemp(n)，可以检查体温n是否正常，函数会返回布尔值。但是，不同的小区有不同的体温检测标准，比如A小区体温合格线是37.1℃，而B小区体温合格线是37.3℃，应该怎么编程呢？

49

闭包用途2 -模拟私有变量

- 题目：请定义一个变量a，要求是能保证这个a只能被进行指定操作（如加1、乘2），而不能进行其他操作，应该怎么编程呢？
- 在Java、C++等语言中，有私有属性的概念，但是JavaScript中只能用闭包来模拟私有变量。

50

使用闭包的注意点

- 不能滥用闭包，否则会造成网页的性能问题，严重时可能导致内存泄露。所谓内存泄漏是指程序中己动态分配的内存由于某种原因未释放或无法释放。
- 所谓内存泄漏是指程序中己动态分配的内存由于某种原因未释放或无法释放。

51

闭包的一道面试题

```
function addCount() {
    var count = 0;
    return function () {
        count = count + 1;
        console.log(count);
    };
}
var fun1 = addCount();
var fun2 = addCount();
fun1();
fun2();
fun2();
fun1();
```

```
1
1
2
2
```

52